

Problem komiwojażera

Katsiaryna Sultanava, 181519

1 czerwca 2026

Problem Komiwożera (TSP – Traveling Salesman Problem)

Cel

Znalezienie najkrótszej, zamkniętej trasy, która startuje z bazy, musi odwiedzić dokładnie raz każde z zaplanowanych miast i bezpiecznie wrócić do punktu wyjścia.



Definicja grafowa

W ujęciu teorii grafów, poszukujemy **Cyклу Hamiltona o minimalnej sumie wag krawędzi** w grafie pełnym ważonym.

Złożoność obliczeniowa

Problem należy do klasy problemów **NP-trudnych**. Liczba możliwych tras rośnie silniowo, co uniemożliwia dokładne obliczenia metodą siłową (Brute-Force).

Założenia matematyczne (Metryczne TSP)

Niech G będzie grafem pełnym, na którym została określona funkcja wagowa $w : E(G) \rightarrow \mathbb{R}^+$. Zakładamy ponadto, że funkcja w spełnia **nierówność trójkąta**, tj.:

$$w(ac) \leq w(ab) + w(bc)$$

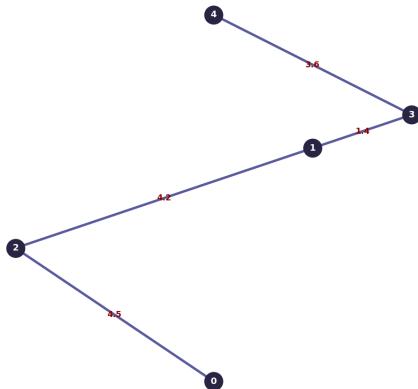
Nierówność trójkąta gwarantuje, że operacja „robienia skrótów” podczas eliminacji duplikatów z przejścia DFS nie wydłuży całkowitej długości trasy.

Złożony problem TSP zostaje rozwiązany w 3 prostych krokach.

Krok pierwszy: Konstrukcja MST

Budujemy Minimalne Drzewo Rozpinające algorytmem Kruskala (szkielet bez cykli łączący wszystkie miasta najmniejszym kosztem).

1. Wyznaczone Drzewo MST



Krok drugi: Przejście DFS

Rejestrujemy pełny spacer w głąb drzewa, uwzględniając powroty ze ślepych zaułków. Podążamy krawędzią w przód. Napotykając ślepy zaułek (liść drzewa), mechanizm backtrackingu rejestruje drogę powrotną do rodzica.

Wierzchołki 1, 2, 3 są odwiedzane wielokrotnie.

Pełna trajektoria (multigraf)

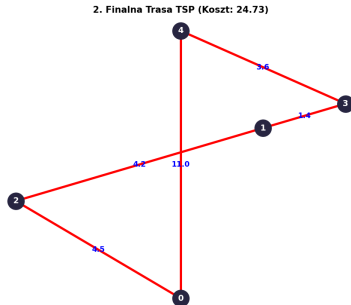
$$0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 0$$

Krok trzeci: Skracanie ścieżki

Przekształcamy spacer w cykl Hamiltona poprzez natychmiastowe pomijanie miast, które już wcześniej odwiedziliśmy.

Wynikowy Cykl Hamiltona (Trasa TSP)

$0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 0$



Algorytm Brute Force

Zasada Przeglądu Wyczerpującego

Algorytm generuje wszystkie możliwe permutacje tras dla pozostałych miast. Dla $n = 5$ i stałego startu w punkcie 0, liczba ścieżek wynosi:

$$(n - 1)! = 4! = 24 \text{ permutacje}$$

Wyznaczenie Optimum Globalnego

Algorytm sprawdza po kolei koszty. Najniższy napotkany koszt zostaje zapamiętany:

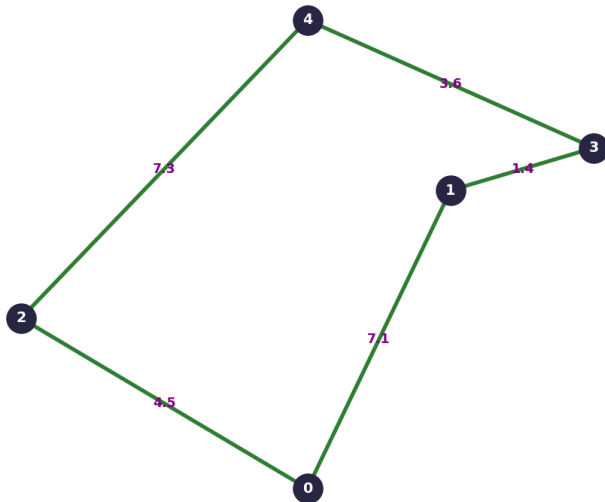
- **Najoptymalniejsza trasa:** $0 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 0$
- **Koszt optymalny (Bezwzględne minimum):** 23.63

Algorytm Brute Force - 24 Permutacje

Nr	Permutacja Trasy	Suma Wag Krawędzi z Macierzy D	Koszt Łączny
1	$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$	$7.07 + 4.24 + 5.66 + 3.61 + 11.00$	31.58
2	$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 0$	$7.07 + 4.24 + 7.28 + 3.61 + 8.25$	30.45
3	$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 0$	$7.07 + 1.41 + 5.66 + 7.28 + 11.00$	32.42
4	$0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 0$	$7.07 + 1.41 + 3.61 + 7.28 + 4.47$	23.84
5	$0 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 0$	$7.07 + 4.12 + 7.28 + 5.66 + 8.25$	32.38
6	$0 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 0$	$7.07 + 4.12 + 3.61 + 5.66 + 4.47$	24.93
7	$0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 0$	$4.47 + 4.24 + 1.41 + 3.61 + 11.00$	24.73 (MST-DFS)
8	$0 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 0$	$4.47 + 4.24 + 4.12 + 3.61 + 8.25$	23.63 (OPTIMUM)
9	$0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 4 \rightarrow 0$	$4.47 + 5.66 + 1.41 + 4.12 + 11.00$	26.66
10	$0 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 0$	$4.47 + 5.66 + 3.61 + 4.12 + 7.07$	24.93
11	$0 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 3 \rightarrow 0$	$4.47 + 7.28 + 4.12 + 1.41 + 8.25$	25.53
12	$0 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1 \rightarrow 0$	$4.47 + 7.28 + 3.61 + 1.41 + 7.07$	23.84
13	$0 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 0$	$8.25 + 1.41 + 4.24 + 7.28 + 11.00$	32.18
14	$0 \rightarrow 3 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 0$	$8.25 + 1.41 + 4.12 + 7.28 + 4.47$	25.53
15	$0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 0$	$8.25 + 5.66 + 4.24 + 4.12 + 11.00$	33.27
16	$0 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 0$	$8.25 + 5.66 + 7.28 + 4.12 + 7.07$	32.38
17	$0 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 0$	$8.25 + 3.61 + 4.12 + 4.24 + 4.47$	24.69
18	$0 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 0$	$8.25 + 3.61 + 7.28 + 4.24 + 7.07$	30.45
19	$0 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$	$11.00 + 4.12 + 4.24 + 5.66 + 8.25$	33.27
20	$0 \rightarrow 4 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$	$11.00 + 4.12 + 1.41 + 5.66 + 4.47$	26.66
21	$0 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 0$	$11.00 + 7.28 + 4.24 + 1.41 + 8.25$	32.18
22	$0 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$	$11.00 + 7.28 + 5.66 + 1.41 + 7.07$	32.42

Algorytm Brute Force - Trasa

Optymalna Trasa Brute Force (Koszt: 23.84)



Algorytm Branch and Bound (B&B)?

Jest to algorytm **dokładny**, gwarantujący znalezienie matematycznego optimum (najkrótszej trasy), który eliminuje wady ślepego przeszukiwania typu Brute Force.

- **Podział (Branching):** Przestrzeń rozwiązań jest dzielona na mniejsze podproblemy w strukturze drzewiastej (decyzje o wyborze konkretnej krawędzi).
- **Ograniczenia (Bounding):** Dla każdego węzła obliczane jest **ograniczenie dolne (Bound)**, czyli minimalny szacowany koszt ukończenia trasy.
- **Odcinanie (Pruning):** Jeśli dolne ograniczenie badanej ścieżki (LB) przekracza koszt dotychczas najlepszego pełnego rozwiązania (UB), **gałąź zostaje natychmiast odcięta**.

Przykład Liczbowy – Pierwsza Redukcja Macierzy Kosztów

Krok 2: Wyznaczanie dolnego ograniczenia (LB) dla punktów

1. Macierz wejściowa:

$$\begin{pmatrix} \infty & 7.07 & 4.47 & 8.25 & 11.00 \\ 7.07 & \infty & 4.24 & 1.41 & 4.12 \\ 4.47 & 4.24 & \infty & 5.66 & 7.28 \\ 8.25 & 1.41 & 5.66 & \infty & 3.61 \\ 11.00 & 4.12 & 7.28 & 3.61 & \infty \end{pmatrix}$$

2. Po redukcji:

$$\begin{pmatrix} \infty & 2.60 & 0 & 3.78 & 4.33 \\ 5.66 & \infty & 2.83 & 0 & 0.51 \\ 0 & 0 & \infty & 1.42 & 0.84 \\ 6.84 & 0 & 4.25 & \infty & 0 \\ 7.39 & 0.51 & 3.67 & 0 & \infty \end{pmatrix}$$

Wszystkie kolumny i wiersze zawierają już zera.

Suma stałych redukcji (LB):

$$R = 4.47 + 1.41 + 4.24 + 1.41 + 3.61 + 2.20 = 17.34$$

Żadna trasa nie będzie krótsza niż nasz koszt zredukowany: **17.34**.

Blokowanie Cykli za pomocą Nieskończoności

Krok 3 i 4: Przejście do potomka $i \rightarrow j$

Aby uniemożliwić przedwczesne zamknięcie pętli lub ponowny start z wykorzystanego miasta, nowa macierz potomka podlega rygorystycznemu blokowaniu.

- 1 Wybrany wierzchołek otrzymuje koszt R .
- 2 **Wiersz i (przodek)** zostaje wypełniony wartościami ∞ .
- 3 **Kolumna j (potomek)** zostaje wypełniona wartościami ∞ .
- 4 **Blokada podcyklu:** Komórka o współrzędnych (j, i) bezwzględnie otrzymuje wartość ∞ .

Zakładamy, że i – przodek wierzchołka j ($i \rightarrow j$). Aby wyznaczyć koszt dla każdego z liści należy zsumować:

- 1 koszt przejścia $i \rightarrow j$ ze zredukowanej macierzy wierzchołka i ,
- 2 koszt przodka i ,
- 3 koszt zredukowania macierzy wierzchołka j .

Wybieramy liść o najmniejszym koszcie i rozwijamy go. Koszt ostatniego liścia jest ogr. górnym (zmieniamy ∞ na nasz koszt). Rozwiązanie jest wyznaczone wtedy, gdy każdy liść ma koszt większy od ograniczenia górnego.

Rozbudowa Drzewa – Wyznaczanie Kosztu Potomka (0 → 1)

Analiza kroku z miasta 0 do miasta 1:

$$\begin{pmatrix} \infty & 2.60 & 0 & 3.78 & 4.33 \\ 5.66 & \infty & 2.83 & 0 & 0.51 \\ 0 & 0 & \infty & 1.42 & 0.84 \\ 6.84 & 0 & 4.25 & \infty & 0 \\ 7.39 & 0.51 & 3.67 & 0 & \infty \end{pmatrix}$$

$$\begin{pmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & 0 & 0.51 \\ 0 & \infty & \infty & 1.42 & 0.84 \\ 6.84 & \infty & 1.42 & \infty & 0 \\ 7.39 & \infty & 0.84 & 0 & \infty \end{pmatrix}$$

- **Koszt krawędzi:** Odczytany z komórki (0, 1) macierzy zredukowanej wynosi **2.60**.
- **Blokowanie:** Wiersz 0 oraz kolumna 1 otrzymują ∞ . Blokada powrotu $(1, 0) = \infty$.
- **Ponowna redukcja:** W zablokowanej macierzy kolumna 2 traci zero (jej minimum wynosi **2.83**). Po odjęciu tej wartości od elementów trzeciej kolumny macierz jest zredukowana.

$$LB(1) = 17.34 + 2.60 + 2.83 = 22.77$$

Wyznaczanie Kosztu Potomka ($0 \rightarrow 2$)

Analiza kroku z miasta 0 do miasta 2:

$$\begin{pmatrix} \infty & 2.60 & 0 & 3.78 & 4.33 \\ 5.66 & \infty & 2.83 & 0 & 0.51 \\ 0 & 0 & \infty & 1.42 & 0.84 \\ 6.84 & 0 & 4.25 & \infty & 0 \\ 7.39 & 0.51 & 3.67 & 0 & \infty \end{pmatrix} \quad \begin{pmatrix} \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 0 & 0.51 \\ \infty & 0 & \infty & 1.42 & 0.84 \\ 1.18 & 0 & \infty & \infty & 0 \\ 1.73 & 0.51 & \infty & 0 & \infty \end{pmatrix}$$

blokujemy wiersz 0, oraz kolumnę 2, (2, 0)-blokada powrotu, nowe minimum: 5.66.

$$LB(2) = 17.34 + 0.00 + 5.66 = 23.00$$

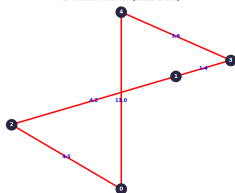
Decyzja algorytmu

Porównując liście, algorytm wybiera gałąź o najniższym koszcie. Ponieważ $22.77 < 23.00$, w pierwszej kolejności system rozpocznie głębszą eksplorację od miasta **1**.

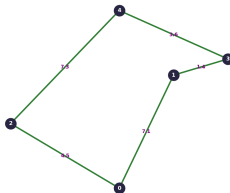
Analiza Wyników dla badanego grafu($n=5$)

Kryterium	MST-DFS	Brute Force	Branch & Bound
Typ algorytmu	Aproksymacyjny	Dokładny	Dokładny
Złożoność	$\mathcal{O}(n^2)$	$\mathcal{O}(n!)$	Zależna od danych
Koszt trasy	24.73	23.84	23.84
Czas pracy	≈ 0.0010 s	≈ 0.0001 s	≈ 0.0007 s

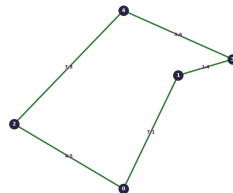
2. Finalna Trasa TSP (Koszt: 24.73)



Optymalna Trasa Brute Force (Koszt: 23.84)

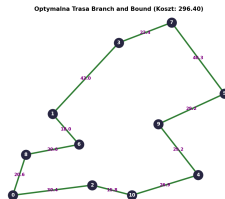
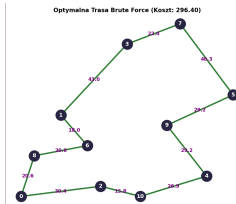
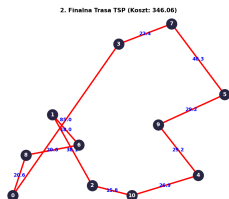


Optymalna Trasa Branch and Bound (Koszt: 23.84)



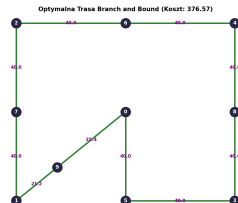
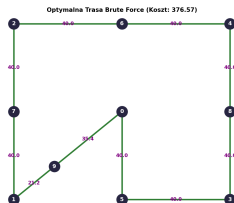
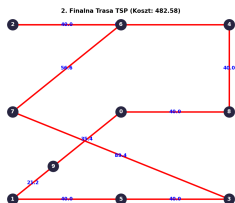
Analiza Wyników dla średniego grafu($n=10$)

Kryterium	MST-DFS	Brute Force	Branch & Bound
Typ algorytmu	Aproksymacyjny	Dokładny	Dokładny
Złożoność	$\mathcal{O}(n^2)$	$\mathcal{O}(n!)$	Zależna od danych
Koszt trasy	346.06	296.40	296.40
Czas pracy	≈ 0.001 s	≈ 8.989 s	≈ 0.173 s



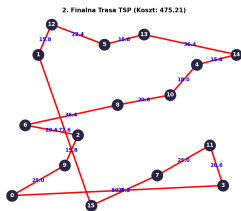
Analiza Wyników dla grafu gwiazda (Jeden punkt centralny, reszta na okręgu ($n=8$))

Kryterium	MST-DFS	Brute Force	Branch & Bound
Typ algorytmu	Aproksymacyjny	Dokładny	Dokładny
Złożoność	$\mathcal{O}(n^2)$	$\mathcal{O}(n!)$	Zależna od danych
Koszt trasy	482.58	376.57	376.57
Czas pracy	≈ 0.0009 s	≈ 0.8204 s	≈ 0.4582 s

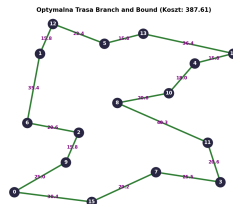


Analiza Wyników dla rozbudowanego grafu (n=16)

Kryterium	MST-DFS	Brute Force	Branch & Bound
Typ algorytmu	Aproksymacyjny	Dokładny	Dokładny
Złożoność	$\mathcal{O}(n^2)$	$\mathcal{O}(n!)$	Zależna od danych
Koszt trasy	475.21	—	387.61
Czas pracy	≈ 0.0015 s	—	≈ 10.018 s



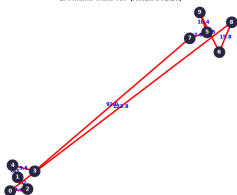
Został zablokowany,
ponieważ czas działania
wyniósł by ≈ 15 dni.



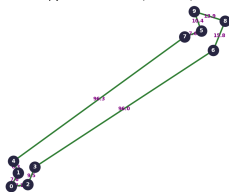
Analiza Wyników dla dwóch odseparowanych grup miast($n=9$)

Kryterium	MST-DFS	Brute Force	Branch & Bound
Typ algorytmu	Aproksymacyjny	Dokładny	Dokładny
Złożoność	$\mathcal{O}(n^2)$	$\mathcal{O}(n!)$	Zależna od danych
Koszt trasy	301.26	270.67	270.67
Czas pracy	≈ 0.001 s	≈ 0.859 s	≈ 0.349 s

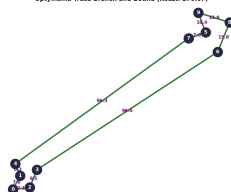
2. Finalna Trasa TSP (Koszt: 301.26)



Optymalna Trasa Brute Force (Koszt: 270.67)



Optymalna Trasa Branch and Bound (Koszt: 270.67)



1. Eksplozja kombinatoryczna ($N = 16$)

Instancja z 16 wierzchołkami udowodniła teoretyczne limity: **Brute Force musiał zostać zablokowany** (szacowany czas pracy to ≈ 15 dni). W tym samym czasie algorytm **Branch and Bound** znalazł rozwiązanie w 10 sekund, a **MST-DFS** w milisekundy.

2. Geometria a wydajność Branch and Bound

- **Graf Klastrowy (Odseparowane grupy):** Algorytm B&B działał niezwykle wydajnie (≈ 0.349 s). Wynika to z silnej redukcji macierzy – wysokie koszty przejść między klastrami pozwoliły na odcięcie całych gałęzi na wczesnym etapie.
- **Graf Gwiazda :** Generował trudniejsze warunki dla B&B (≈ 0.458 s dla mniejszego $N = 8$). Podobne odległości krawędzi utrudniają jednoznaczne odcinanie nieefektywnych poddrzew.

- 1 **Brute Force:** Przydatny wyłącznie jako referencyjny punkt odniesienia dla małych zadań ($N \leq 10$). Powyżej tej granicy staje się bezużyteczny przez silniowy wzrost złożoności.
- 2 **Branch and Bound :** Gwarantuje 100% optimum matematycznego, a dzięki inteligentnemu odcinaniu macierzowemu przesuwaa praktyczną barierę obliczalności do średnich wielkości grafów.
- 3 **MST-DFS:** Najlepsza wydajnościowo (zawsze poniżej kilku milisekund, złożoność $\mathcal{O}(n^2)$). Ceną za natychmiastowy wynik jest błąd aproksymacji, wynoszący w testach od kilku do kilkunastu procent.

Dziękuję za uwagę!